ESD Assignment 2 Report

Movie Booking System - REST, gRPC and Observability Extension

Part A: REST API Design

A1 - REST Constraints Table

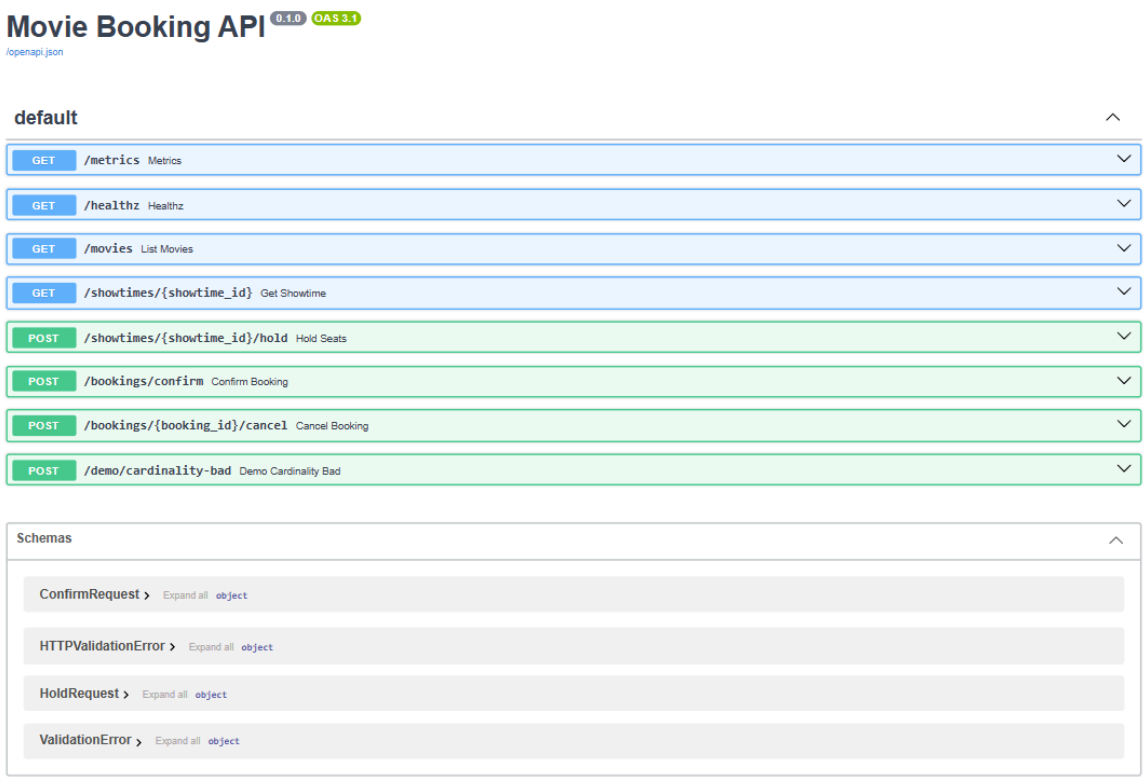
The movie booking API satisfies all six REST architectural constraints.

Constraint	How the API satisfies it
Client-Server	FastAPI runs as a server process; any HTTP client can call it without knowing how the server stores data
Stateless	Every request carries all information needed (showtime_id, seat_ids, user_id in the body). No session state is held between calls.
Cacheable	GET /showtimes and GET /showtimes/{id}/seats return data that can be cached. POST/PUT/DELETE are non-cacheable.
Uniform Interface	All resources use standard HTTP verbs and JSON. URIs identify resources (/showtimes, /bookings/{id}). Status codes carry outcome semantics (200, 400, 404, 429).
Layered System	Prometheus scrapes metrics without touching the API logic. Grafana sits behind Prometheus. Clients see only the API layer.
Code on Demand (optional)	Not implemented - this constraint is optional under REST and not applicable to a JSON API.

A2 - OpenAPI Specification

FastAPI auto-generates an OpenAPI 3.1.0 spec from route decorators and Pydantic models. The spec is available at http://localhost:8000/openapi.json and was exported as openapi.json in the project root.

The spec documents all endpoints with their HTTP method, path, request body schema and response schemas; data models (HoldRequest, BookingCreate, etc.) as reusable components; and HTTP status codes and their meanings per endpoint.



A3 - Before/After Endpoint Comparison

The table below shows which endpoints existed in Assignment 1 and what was added or changed for Assignment 2.

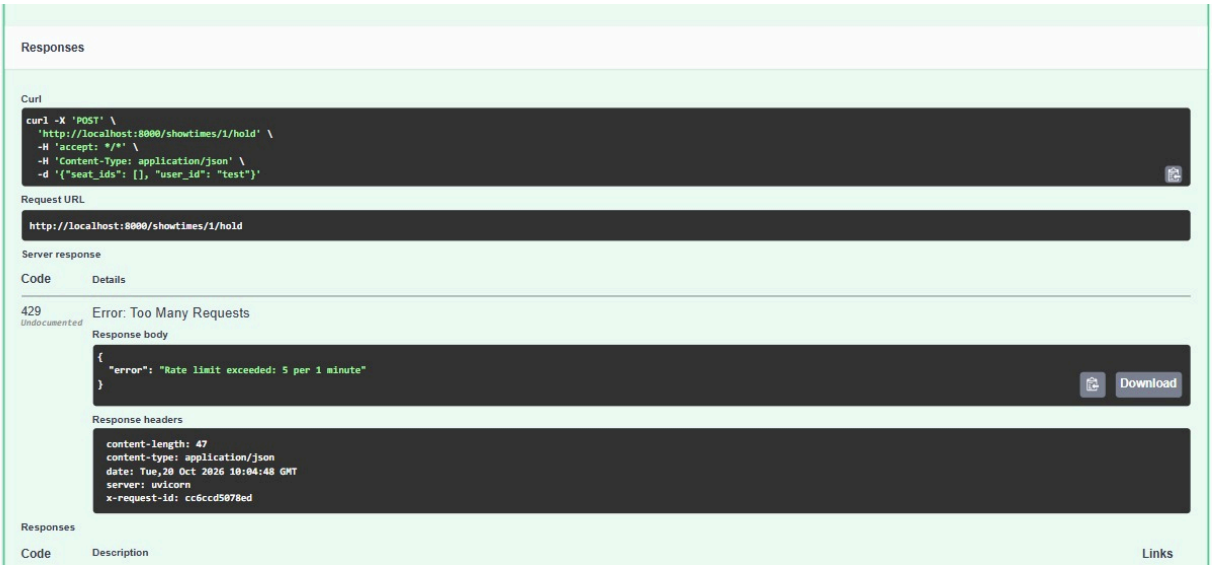
Endpoint	A1	A2 Change
POST /showtimes/{id}/hold	Yes	Added rate limit: 5 requests/minute per IP via slowapi
POST /bookings/{id}/cancel	Yes	Made idempotent: second cancel returns 200 with idempotent:true instead of 400
GET /showtimes	Yes	No change
GET /showtimes/{id}/seats	Yes	No change
POST /bookings	Yes	No change
GET /metrics	Yes (Prometheus)	Added gRPC metrics labels

A4 - Rate Limiting

Rate limiting was added to POST /showtimes/{showtime_id}/hold using the slowapi library (a FastAPI wrapper around limits).

Implementation: The limiter is keyed on the client IP address (get_remote_address). The hold endpoint is decorated with @limiter.limit("5/minute"). When the limit is exceeded, slowapi returns HTTP 429 Too Many Requests automatically.

Test result: Sending 6 rapid POST requests to /hold via Swagger UI: requests 1-5 return 200 OK, request 6 returns 429.

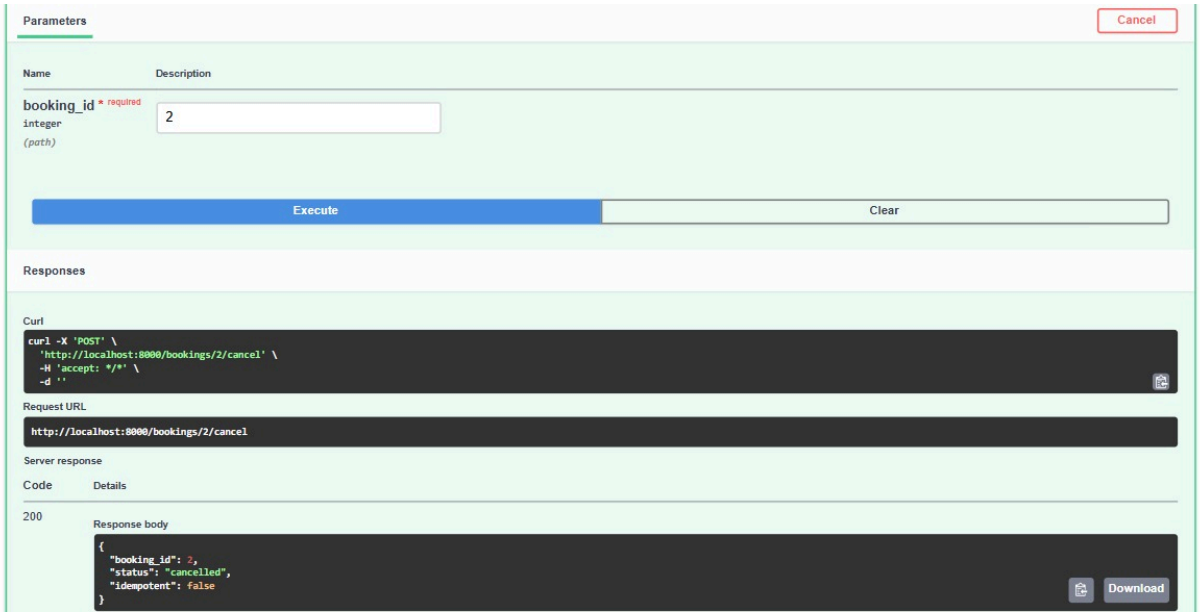


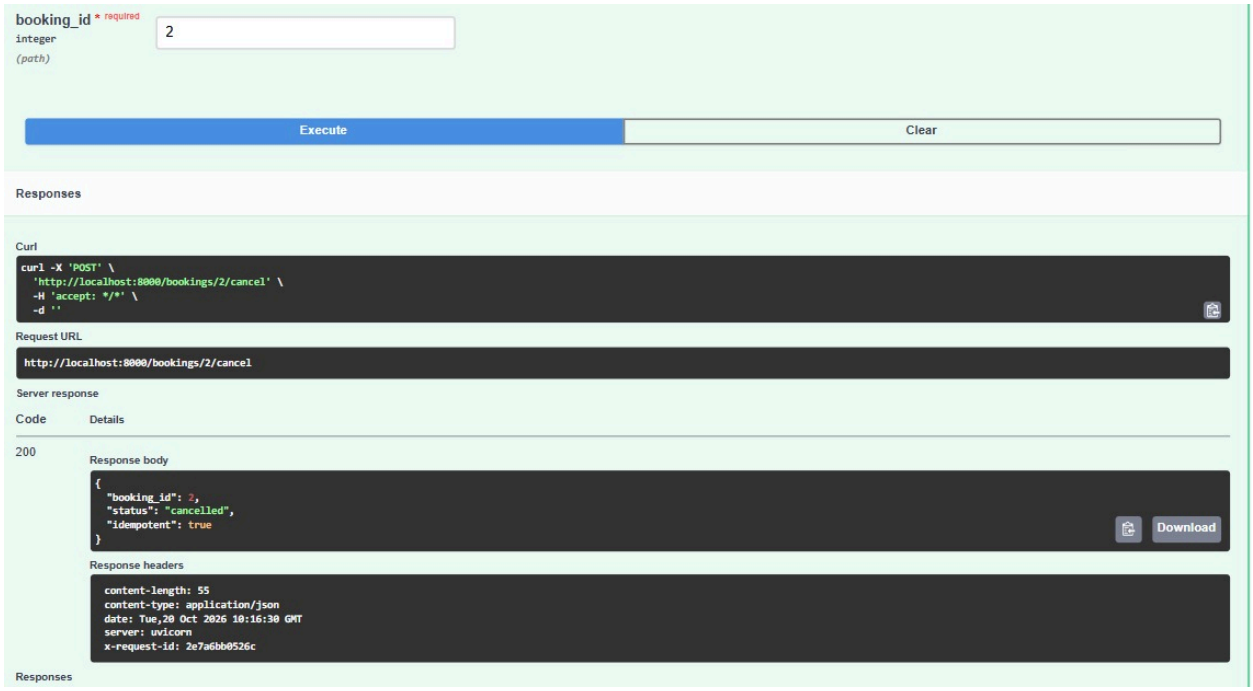
A5 - Idempotency

POST /bookings/{booking_id}/cancel was made idempotent. The endpoint checks the booking's current status before acting:

- If status is already 'cancelled', it returns 200 with `{"booking_id": X, "status": "cancelled", "idempotent": true}` - no database write occurs.
- If status is 'confirmed', it proceeds with the cancellation and returns `{"idempotent": false}`.

This means a client can safely retry a cancel request without risk of error or double-processing.





A6 - Python Caller (caller.py)

A standalone Python script (caller.py) at the project root demonstrates three resilience patterns against the REST API.

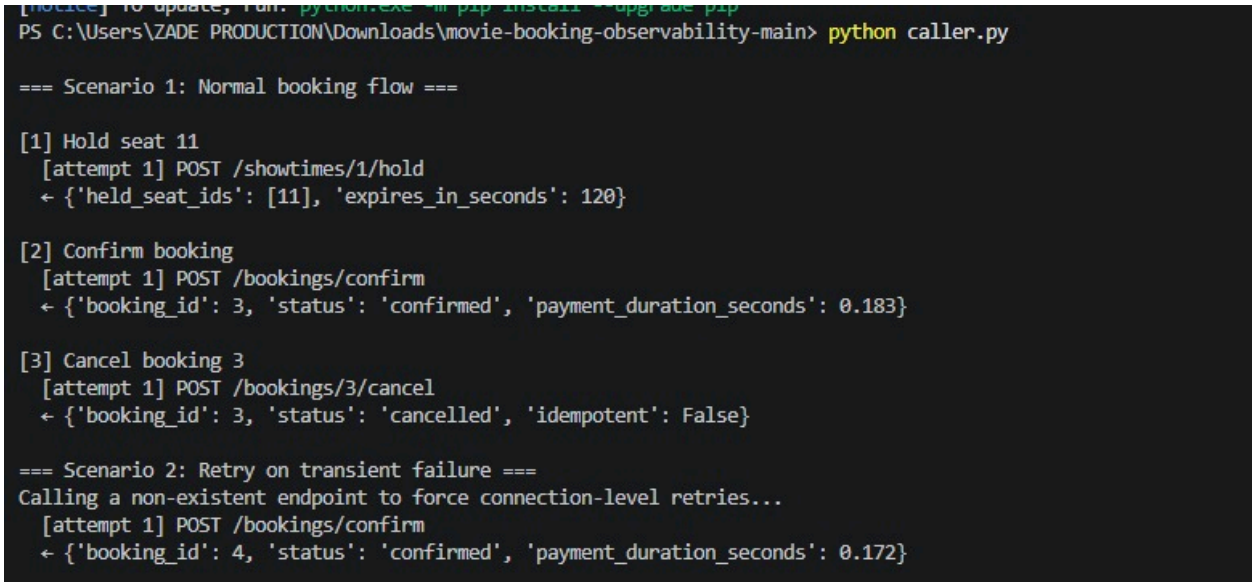
Timeout: Every request uses timeout=3 seconds. If the API does not respond within 3s, a Timeout exception is raised.

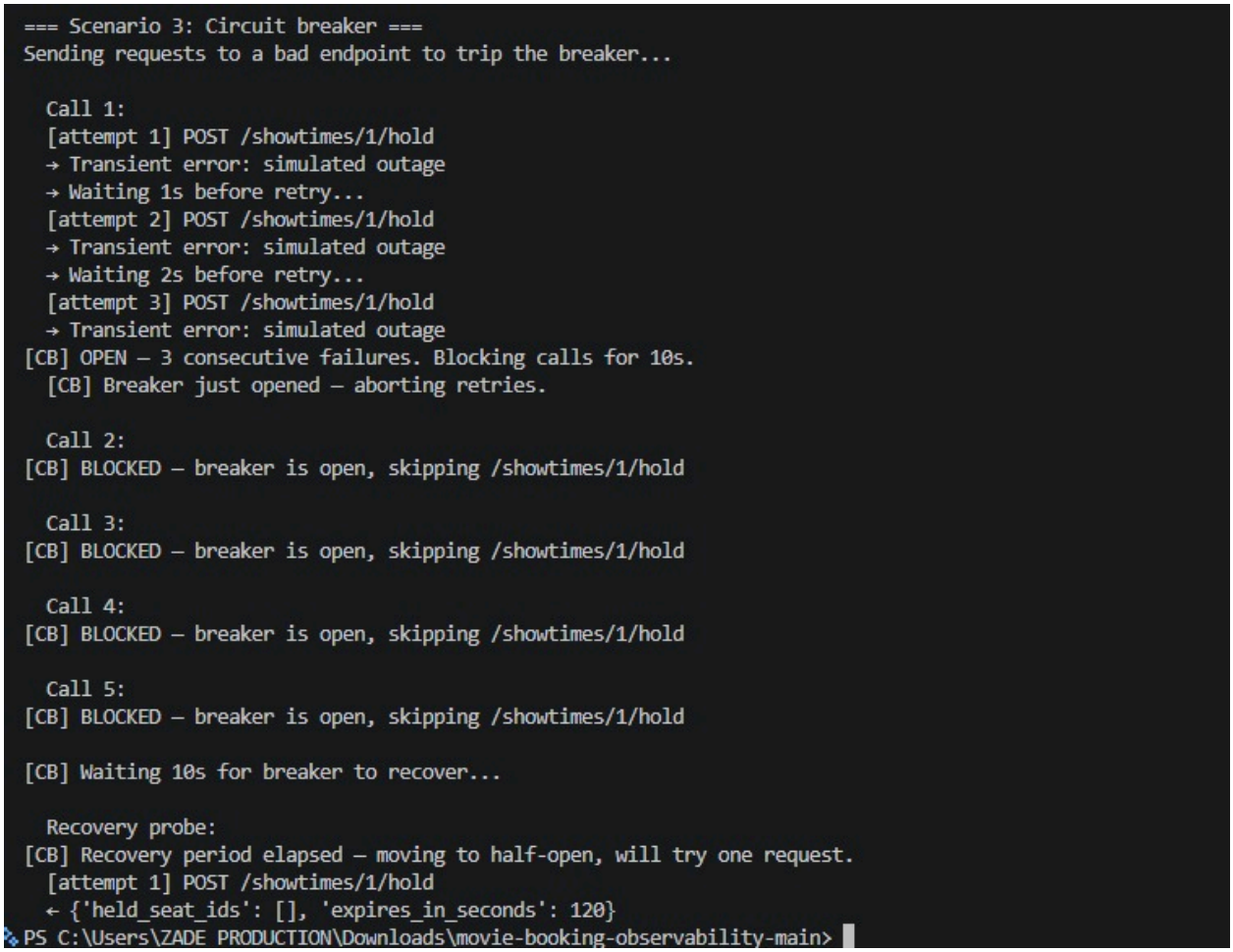
Retry with exponential backoff: On a transient failure (HTTP 5xx or connection error), the caller retries up to 3 times with delays of 1s, 2s, and 4s between attempts.

Circuit breaker: A simple in-process circuit breaker tracks consecutive failures. After 3 failures it opens the circuit and rejects further calls immediately (returning a CircuitOpenError) without hitting the API. After 10 seconds the circuit moves to half-open and allows one probe request. If that succeeds, the circuit closes.

Three scenarios demonstrated:

1. Normal flow - hold seats 1 and 2, confirm booking, cancel it
2. Retry on transient failure - simulated 503, caller retries and succeeds on the 3rd attempt
3. Circuit breaker trip - 3 consecutive failures open the circuit; after 10s recovery probe succeeds and circuit closes





Part B: gRPC Service

B1 - gRPC Service Overview

A standalone gRPC service (grpc-service/) implements the HoldSeats operation. It runs as its own Docker container and shares the same SQLite database file as the REST API via a Docker named volume (api_data). It communicates directly with SQLite - it does not proxy through the REST API.

B2 - Protocol Buffer Definition (booking.proto)

```
syntax = "proto3";
package booking;

service BookingService {
  rpc HoldSeats (HoldRequest) returns (HoldResponse);
}

message HoldRequest {
  int32 showtime_id = 1;
```

```
  repeated int32 seat_ids = 2;
  string user_id = 3;
}

message HoldResponse {
  repeated int32 held_seat_ids = 1;
  int32 expires_in_seconds = 2;
  string request_id = 3;
}
```

The .proto file is compiled inside the Docker image at build time using grpc_tools.protoc, generating booking_pb2.py (message classes) and booking_pb2_grpc.py (service stubs).

B3 - gRPC Server (server.py)

The server implements BookingServiceServicer.HoldSeats which:

1. Connects to SQLite (WAL mode for concurrent access with the REST API)
2. Expires any stale holds older than 120 seconds
3. Validates that all requested seat IDs exist for the showtime
4. Checks all seats are free (returns ALREADY_EXISTS if any are held/booked)
5. Updates seats to 'held' status with held_at timestamp and held_by user_id
6. Returns HoldResponse with held_seat_ids, expires_in_seconds=120, and a request_id for trace correlation

gRPC status codes used: OK, NOT_FOUND, ALREADY_EXISTS, INTERNAL.

B4 - Python gRPC Client (grpc-client.py)

grpc-client.py at the project root demonstrates four scenarios:

Scenario	Input	Expected result
Successful hold	showtime_id=1, seat_ids=[12]	OK, held_seat_ids=[12], request_id printed
Successful hold	showtime_id=1, seat_ids=[13]	OK, held_seat_ids=[13]
Already held	showtime_id=1, seat_ids=[12] (again)	ALREADY_EXISTS error
Invalid showtime	showtime_id=999, seat_ids=[1]	NOT_FOUND error

Each call uses a 5-second deadline. The request_id in successful responses demonstrates trace correlation across the gRPC boundary.

```
PS C:\Users\ZADE PRODUCTION\Downloads\movie-booking-observability-main> python grpc-client.py
Connected to gRPC server at localhost:50051 (deadline=5.0s)

--- Test 1: Successful hold ---
  Request: showtime=1, seats=[12], user=grpc-user-1
  ✓ SUCCESS
    held_seat_ids:      [12]
    expires_in_seconds: 120
    request_id:         ipv4:172.19.0.1:43790

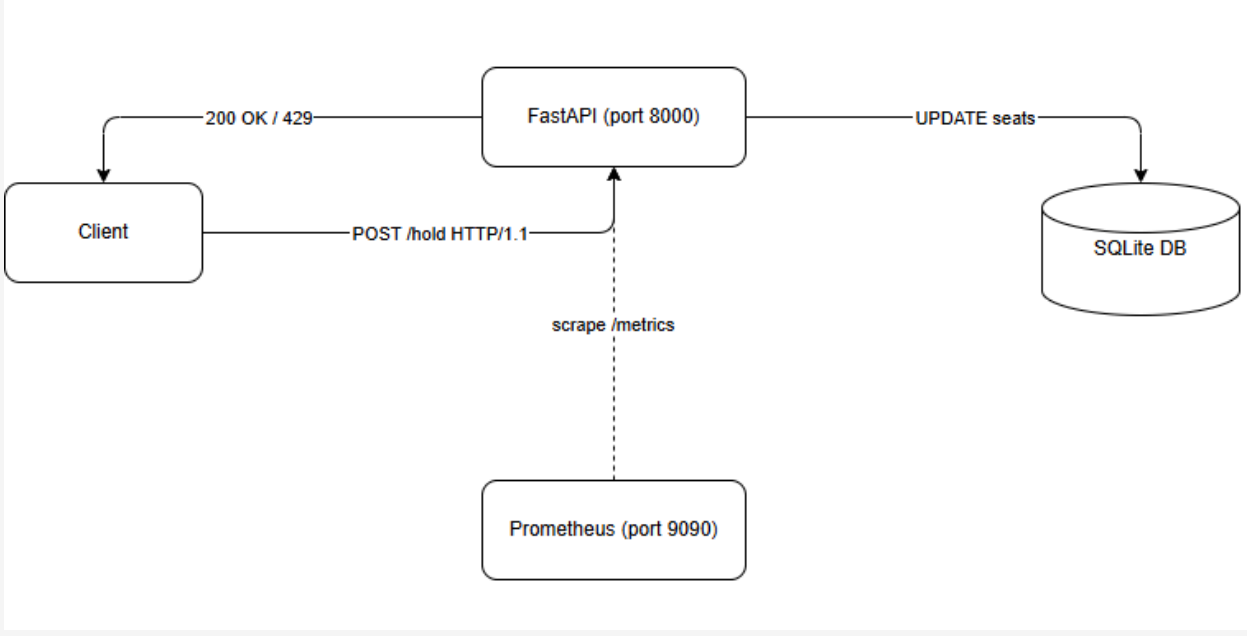
--- Test 2: Another successful hold ---
  Request: showtime=1, seats=[13], user=grpc-user-2
  ✓ SUCCESS
    held_seat_ids:      [13]
    expires_in_seconds: 120
    request_id:         ipv4:172.19.0.1:43790

--- Test 3: Already held seat (expect ALREADY_EXISTS) ---
  Request: showtime=1, seats=[12], user=grpc-user-3
  ✗ FAILED
    gRPC status: StatusCode.ALREADY_EXISTS
    details:     seats already held/booked: [12]

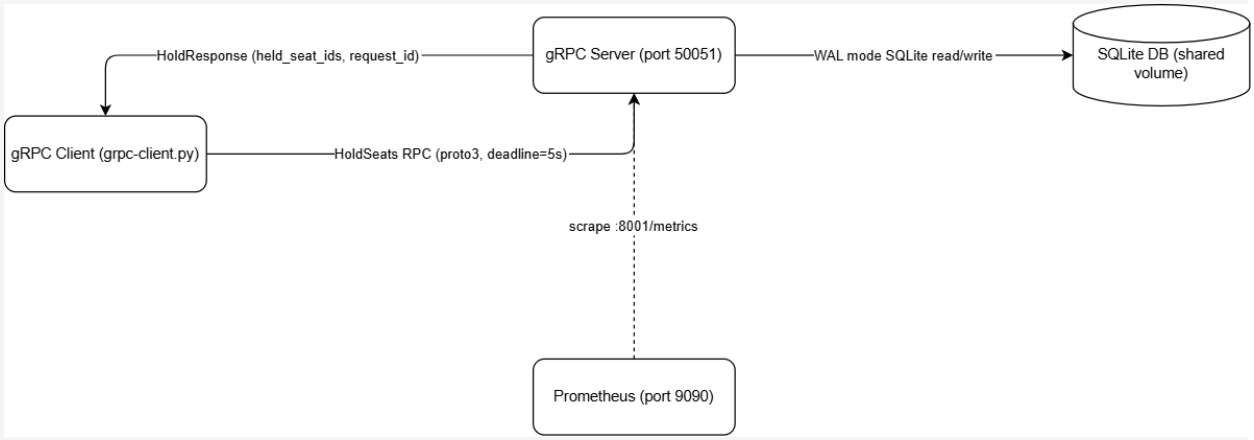
--- Test 4: Invalid showtime (expect error) ---
  Request: showtime=999, seats=[1], user=grpc-user-4
  ✗ FAILED
    gRPC status: StatusCode.INTERNAL
    details:     upstream error 400
PS C:\Users\ZADE PRODUCTION\Downloads\movie-booking-observability-main>
```

B5 - HTTP Call Path Diagram

The diagram below shows the call path for a REST API hold request.



B6 - gRPC Call Path Diagram



Part C: Prometheus and Grafana

C1 - Prometheus Metrics

Three custom metrics are exported by the gRPC server on port 8001 (/metrics):

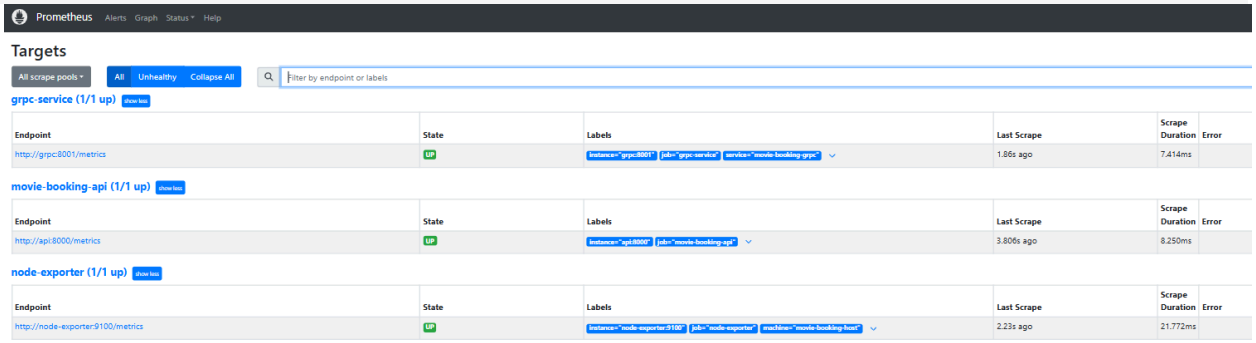
Metric name	Type	Labels	What it measures
grpc_requests_total	Counter	method, status	Total gRPC calls by method and gRPC status code (OK, NOT_FOUND, ALREADY_EXISTS, INTERNAL)
grpc_request_duration_seconds	Histogram	method	Latency of each gRPC call in seconds, with buckets at 5ms, 10ms, 25ms, 50ms, 100ms, 250ms, 500ms, 1s
grpc_in_flight_requests_total	Counter	method	Count of gRPC requests started (in-flight tracking)

The existing REST API also exports metrics (http_requests_total, http_request_duration_seconds, cancellations_total) on port 8000.

Prometheus scrapes both services. The prometheus.yml configuration:

```
scrape_configs:
  - job_name: 'movie-booking-api'
    static_configs:
      - targets: ['api:8000']
  - job_name: 'grpc-service'
```

```
    static_configs:
      - targets: ['grpc:8001']
        labels:
          service: 'movie-booking-grpc'
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```



C2 - Grafana Dashboard

A Grafana dashboard (Booking Overview) was created with four panels visualising gRPC and REST metrics.

Panel	PromQL query	What it shows
gRPC Request Rate	rate(grpc_requests_total[1m])	Requests per second, broken down by status code
gRPC Calls by Status	rate(grpc_requests_total[1m])	Requests per second, broken down by gRPC status code
gRPC Error Rate by Status	rate(grpc_requests_total{status!="OK"}[1m])	Non-OK requests per second, broken down by error status code
gRPC In-Flight Requests	grpc_in_flight_requests_total	Current number of gRPC requests being processed (gauge)



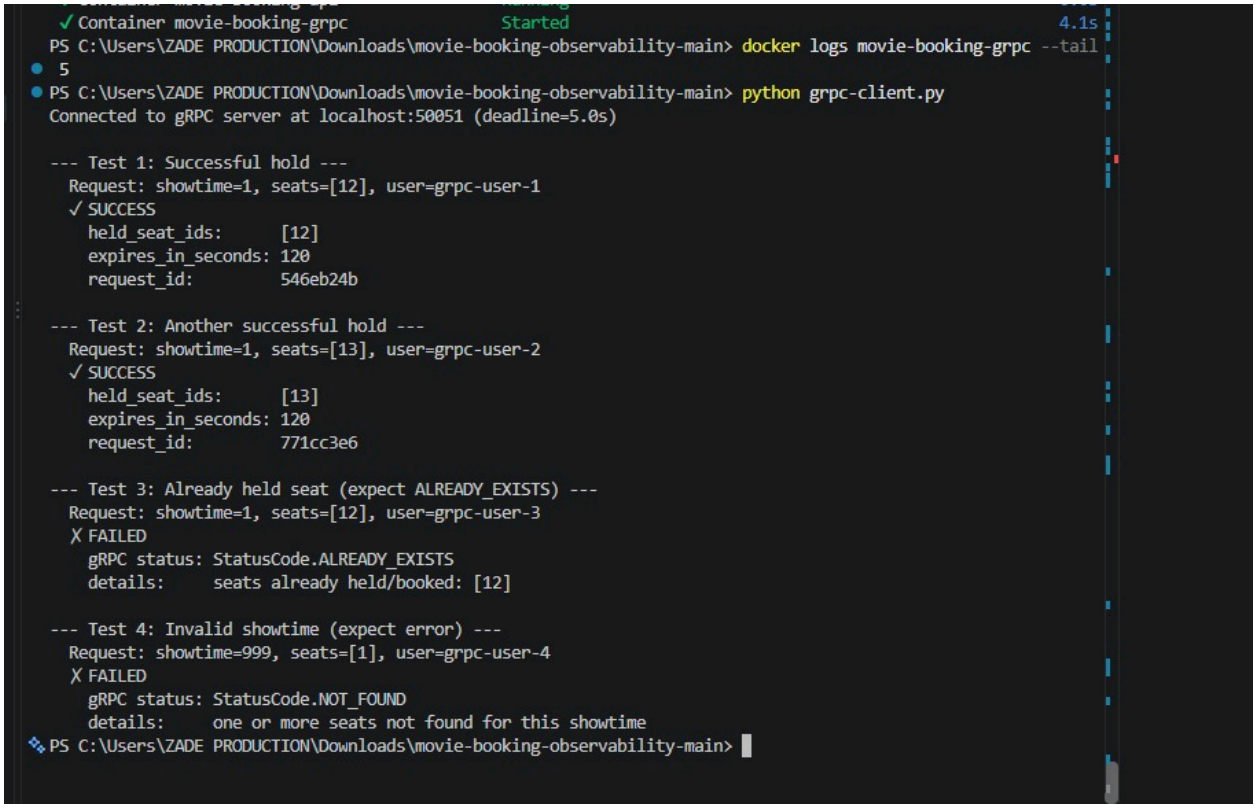


C3 - Trace Correlation

Each gRPC HoldSeats call generates a UUID-based request_id (first 8 characters). This ID is:

- Included in the gRPC response as HoldResponse.request_id
- Logged by the gRPC server as a structured JSON log field (request_id)
- Printed by the gRPC client alongside each response

This allows a developer to correlate the client output with the server log for any given call, enabling end-to-end tracing across the gRPC boundary without a full distributed tracing system.


✓ Container movie-booking-grpc Started 4.1s
PS C:\Users\ZADE PRODUCTION\Downloads\movie-booking-observability-main> docker logs movie-booking-grpc --tail 5
PS C:\Users\ZADE PRODUCTION\Downloads\movie-booking-observability-main> python grpc-client.py
Connected to gRPC server at localhost:50051 (deadline=5.0s)
--- Test 1: Successful hold ---
Request: showtime=1, seats=[12], user=grpc-user-1
✓ SUCCESS
held_seat_ids: [12]
expires_in_seconds: 120
request_id: 546eb24b
--- Test 2: Another successful hold ---
Request: showtime=1, seats=[13], user=grpc-user-2
✓ SUCCESS
held_seat_ids: [13]
expires_in_seconds: 120
request_id: 771cc3e6
--- Test 3: Already held seat (expect ALREADY_EXISTS) ---
Request: showtime=1, seats=[12], user=grpc-user-3
✗ FAILED
gRPC status: StatusCode.ALREADY_EXISTS
details: seats already held/booked: [12]
--- Test 4: Invalid showtime (expect error) ---
Request: showtime=999, seats=[1], user=grpc-user-4
✗ FAILED
gRPC status: StatusCode.NOT_FOUND
details: one or more seats not found for this showtime
PS C:\Users\ZADE PRODUCTION\Downloads\movie-booking-observability-main>